

Deep Learning Notes: Binary Classification & Notation

1. Core Concepts

- **Goal:** To learn a classifier that inputs a feature vector x and predicts a label y .
 - $y = 1$: Positive class (e.g., "Cat").
 - $y = 0$: Negative class (e.g., "Non-cat").
- **Binary Classification:** The task of classifying elements into two distinct groups.

2. Image Representation

- **RGB Matrices:** Images are stored as three separate matrices representing Red, Green, and Blue color channels.
- **Feature Vector Unrolling:** To create an input vector x , pixel intensity values from all three matrices are "unrolled" into a single long vector.
 - **Calculation:** If an image is 64×64 pixels:
 - Total dimensions = $64 \times 64 \times 3$ (RGB channels).
 - $n_x = 12,288$.
 - **Notation:** n_x (or simply n) represents the dimension of the input feature vector x .

3. Dataset Notation

- **Single Training Example:** (x, y)
 - $x \in \mathbb{R}^{n_x}$
 - $y \in \{0, 1\}$
- **Training Set Size:** m denotes the number of training examples.
 - Sometimes written as m_{train} vs m_{test} .
- **Training Set Collection:** $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

4. Matrix Organization (Crucial for Implementation)

To efficiently process data without for loops, data is stacked into matrices.

- **Input Matrix X :**
 - Formed by stacking input vectors $x^{(i)}$ as **columns**.

$$\circ X = \begin{bmatrix} \vdots & \vdots & \vdots \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ \vdots & \vdots & \vdots \end{bmatrix}$$

- **Shape:** (n_x, m) — Rows are features, columns are examples.
- *Python command:* `X.shape` returns (n_x, m) .

- **Output Matrix Y :**

- Formed by stacking labels $y^{(i)}$ as **columns**.
- $Y = [y^{(1)} \quad y^{(2)} \quad \dots \quad y^{(m)}]$
- **Shape:** $(1, m)$
- *Python command:* `Y.shape` returns $(1, m)$.

Note: This "column-stacking" convention differs from some other machine learning courses but makes neural network implementation significantly easier.

5. Neural Network Programming Basics

- **Vectorization:** The practice of processing the entire training set (m examples) simultaneously, avoiding explicit `for` loops.
- **Computation Structure:**
 - **Forward Propagation:** Computing outputs from inputs.
 - **Backward Propagation:** Computing gradients for learning.

Logistic Regression Model

1. Problem Definition

- **Algorithm Type: Logistic Regression** is a learning algorithm used for **binary classification** problems where output labels y are either 0 or 1.
- **Goal:** Given an input feature vector x (e.g., an image), prediction $\hat{y}$ is the estimate of y .
- **Probabilistic Interpretation:** Formally, we want $\hat{y}$ to represent the probability that $y = 1$ given the input features x :

$$\hat{y} = P(y = 1|x)$$
(Example: "What is the chance that this image is a cat?")

2. Model Representation

- **Parameters:**
 - w : An n_x -dimensional vector (weights).
 - b : A real number (bias).
- **Linear Output Issue:** Using a standard linear function like $\hat{y} = w^T x + b$ is unsuitable for binary classification because the output can be much greater than 1 or negative, violating probability rules.
- **Logistic Output:** To ensure the output stays between 0 and 1, we apply the **Sigmoid function** to the linear combination.
 - Let $z = w^T x + b$
 - Then, $\hat{y} = \sigma(z)$

3. The Sigmoid Function

- **Formula:**
$$\sigma(z) = \frac{1}{1+e^{-z}}$$
- **Behavior:**
 - **If z is very large:** $e^{-z} \approx 0$, so $\sigma(z) \approx \frac{1}{1} = 1$.
 - **If z is very small (large negative):** e^{-z} becomes huge, so $\sigma(z) \approx \frac{1}{\text{huge}} \approx 0$.
 - **At $z = 0$:** $\sigma(z) = 0.5$.
- **Role:** It maps any real-valued number z smoothly to the range $(0, 1)$, making it suitable for probability estimation.

4. Notation Convention (Course Specific)

- **Separation of Parameters:** In this course, parameters w and b are kept separate.
 - b corresponds to the intercept term.
 - **Alternative Convention (Not used here):**
 - Some other courses add an extra feature $x_0 = 1$ and define a combined parameter vector θ (where θ_0 is b and $\theta_1 \dots \theta_{n_x}$ is w).
 - *Note:* This course **avoids** the $\theta^T x$ convention to simplify neural network implementation later.
-

Logistic Regression Cost Function

1. Training Context & Notation

- **Goal:** To find parameters w and b such that the output $\hat{y}^{(i)}$ closely matches the ground truth label $y^{(i)}$ for valid training examples.
- **Notation:**
 - The superscript (i) denotes data associated with the i^{th} training example.
 - Prediction: $\hat{y}^{(i)} = \sigma(w^T x^{(i)} + b)$
 - True Label: $y^{(i)}$

2. The Loss Function (Single Example)

The **Loss Function**, denoted as $L(\hat{y}, y)$, measures the discrepancy between the prediction $\hat{y}$ and the true label y for a **single training example**.

- **Why not Squared Error?**
 - In linear regression, we often use $L(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$.
 - In logistic regression, using squared error results in a **non-convex** optimization problem (multiple local optima). This makes Gradient Descent inefficient or unable to find the global optimum.
 - Therefore, we require a different loss function that guarantees a **convex** problem.
- **Logistic Regression Loss Function:**
$$L(\hat{y}, y) = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$$
- **Intuition behind the formula:**
 - **Case 1: If $y = 1$:**
 - The second term $(1 - y) \log(1 - \hat{y})$ becomes 0.
 - $L = -\log(\hat{y})$.
 - To minimize Loss, we must minimize $-\log(\hat{y})$, which means maximizing $\hat{y}$.
 - Since $\hat{y}$ is a probability ($\in [0, 1]$), we want $\hat{y} \approx 1$.
 - **Case 2: If $y = 0$:**
 - The first term $y \log(\hat{y})$ becomes 0.
 - $L = -\log(1 - \hat{y})$.
 - To minimize Loss, we must maximize $(1 - \hat{y})$, which implies minimizing $\hat{y}$.
 - We want $\hat{y} \approx 0$.

3. The Cost Function (Entire Training Set)

The **Cost Function**, denoted as $J(w, b)$, measures the performance of the model on the **entire training set** of m examples. It is the average of the sum of individual losses.

- **Formula:**

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

- **Expanded Formula:**

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

4. Summary of Objectives

- **Loss Function (L):** Applied to a single training example.
 - **Cost Function (J):** Cost of the parameters over the entire training set.
 - **Optimization Goal:** Find parameters w and b that minimize the overall Cost Function $J(w, b)$.
-

Gradient Descent Algorithm

1. Objective

- **Goal:** To learn the optimal parameters w and b that minimize the cost function $J(w, b)$ on the training set.
- **Recap:**
 - Loss function: Measures error on a single example.
 - Cost function (J): Average loss over the entire training set.
 - We want to find w, b such that $J(w, b)$ is as small as possible.

2. Visualization and Convexity

- **Parameter Space:** Imagine a 3D plot where the horizontal axes represent parameters w and b , and the vertical axis represents the cost $J(w, b)$.
- **Convexity:** The cost function for logistic regression is **convex**, meaning it is shaped like a single large bowl.
 - **Significance:** Being convex guarantees that there is only one global optimum (minimum). There are no local optima to get stuck in, unlike non-convex functions (which look wavy with multiple valleys).
- **Initialization:** Due to convexity, you can initialize w and b almost anywhere

(usually 0 or random values) and the algorithm will converge to the same global minimum.

3. The Algorithm

Gradient Descent works by starting at an initial point and repeatedly taking steps in the direction of the **steepest descent** (downhill) until convergence.

Update Rules

Repeat the following steps until the algorithm converges:

$$w := w - \alpha \frac{\partial J(w,b)}{\partial w}$$
$$b := b - \alpha \frac{\partial J(w,b)}{\partial b}$$

- $:=$: Represents assignment (update the value).
- α (**Alpha**): The **Learning Rate**. This controls the size of the step taken on each iteration.
- **Derivative Term** ($\frac{\partial J}{\partial w}$): Represents the slope or the direction of the update.

Intuition on Derivatives (Slope)

- The derivative represents the slope of the function at the current point.
- **Positive Slope**: If the derivative is positive (sloping up to the right), the update rule subtracts from w , moving it to the left (towards the minimum).
- **Negative Slope**: If the derivative is negative (sloping down to the right), the update rule subtracts a negative number (adds), moving w to the right (towards the minimum).

-

4. Notation & Implementation Details

- **Calculus Notation (d vs ∂):**
 - d : Used when differentiating a function of a **single variable** (e.g., $J(w)$).
 - ∂ (**partial derivative symbol**): Used when differentiating a function of **two or more variables** (e.g., $J(w, b)$).
 - *Note*: Mathematically they represent the same concept (slope) for our purposes. The distinction is purely a formality in calculus notation.
- **Code Convention:**
 - When implementing this in code (e.g., Python), we use variable names to represent the derivative terms:
 - dw represents $\frac{\partial J(w,b)}{\partial w}$

- db represents $\frac{\partial J(w,b)}{\partial b}$
 - **Code Update Step:** $w = w - \text{alpha} * dw$
-

Calculus and Derivatives Intuition

1. The Intuitive Definition

- **Derivative = Slope:** While "derivative" can sound like a complex mathematical term, it simply refers to the **slope** of a function.
- **The "Nudge" Method:** To understand the derivative at a specific point for a variable a :
 1. Take the current value of a .
 2. "Nudge" a slightly to the right by a tiny amount (e.g., increase by 0.001).
 3. Observe how much the function output $f(a)$ changes.
 4. The derivative is the ratio of the **change in output** to the **change in input**.

2. Linear Example: $f(a) = 3a$

Consider the function for a straight line, $f(a) = 3a$.

- **Case Study A: $a = 2$**
 - **Initial State:** If $a = 2$, then $f(a) = 6$.
 - **The Nudge:** Increase a to 2.001 (a change of +0.001).
 - **Result:** $f(2.001) = 3 \times 2.001 = 6.003$.
 - **Change in Output:** The function value increased by 0.003.
 - **Calculation:** $\frac{\text{Change in } f(a)}{\text{Change in } a} = \frac{0.003}{0.001} = 3$.
 - **Meaning:** When you nudge a , the function $f(a)$ goes up by **3 times** the size of the nudge.
- **Case Study B: $a = 5$**
 - **Initial State:** If $a = 5$, then $f(a) = 15$.
 - **The Nudge:** Increase a to 5.001.
 - **Result:** $f(5.001) = 15.003$.
 - **Calculation:** Slope is still $\frac{0.003}{0.001} = 3$.
- **Conclusion:** For a straight line (linear function), the derivative is **constant** everywhere. Regardless of the value of a , the slope is always 3.

3. Formal Notation and Definition

- **Notation:** The derivative of function $f(a)$ with respect to variable a is written as:
 - $\frac{df(a)}{da} = 3$
 - Alternatively: $\frac{d}{da} f(a) = 3$
- **Formal Definition:** While the intuitive explanation uses a small number like 0.001, the strict mathematical definition assumes the nudge is **infinitesimal** (infinitely small).

4. Relevance to Deep Learning

- **Depth of Knowledge:** You do **not** need to be a calculus expert to effectively apply deep learning.
 - **Application:** In programming neural networks, calculus operations are often encapsulated in "Forward" and "Backward" functions.
 - **Requirement:** The primary requirement is the intuitive understanding that the derivative represents how much the output changes relative to a change in the input (slope).
-

Derivatives: Non-Linear Examples

1. Concept: Variable Slope

Unlike linear functions (straight lines) where the slope is constant, non-linear functions (curved lines) have slopes that **change** depending on where you are on the curve.

2. Example 1: $f(a) = a^2$

- **The Function:** A parabolic curve.
- **Case A: $a = 2$**
 - Current value: $f(2) = 2^2 = 4$.
 - **Nudge:** Increase a to 2.001.
 - New value: $f(2.001) \approx 4.004$.
 - **Observation:** Increasing a by 0.001 increased $f(a)$ by roughly 0.004.
 - **Slope:** $4 \times$ the input change.
 - **Derivative at $a = 2$:** $\frac{d}{da} f(a) = 4$.
- **Case B: $a = 5$**

- Current value: $f(5) = 5^2 = 25$.
- **Nudge:** Increase a to 5.001.
- New value: $f(5.001) \approx 25.010$.
- **Observation:** Increasing a by 0.001 increased $f(a)$ by roughly 0.010.
- **Slope:** $10 \times$ the input change.
- **Derivative at $a = 5$:** $\frac{d}{da} f(a) = 10$.
- **The Formula:**
 - Calculus textbooks state: $\frac{d}{da}(a^2) = 2a$.
 - **Verification:**
 - At $a = 2$, slope = $2(2) = 4$.
 - At $a = 5$, slope = $2(5) = 10$.

3. Example 2: $f(a) = a^3$

- **The Function:** Cubic curve.
- **The Formula:** $\frac{d}{da}(a^3) = 3a^2$.
- **Verification at $a = 2$:**
 - Predicted Slope: $3(2)^2 = 3(4) = 12$.
 - Numerical Check:
 - $2^3 = 8$.
 - $2.001^3 \approx 8.012$.
 - Change is 0.012, which is exactly 12×0.001 .

4. Example 3: $f(a) = \ln(a)$ (Natural Log)

- **The Function:** Logarithmic curve.
- **The Formula:** $\frac{d}{da} \ln(a) = \frac{1}{a}$.
- **Verification at $a = 2$:**
 - Predicted Slope: $\frac{1}{2} = 0.5$.
 - Numerical Check:
 - $\ln(2) \approx 0.69315$.
 - $\ln(2.001) \approx 0.69365$.
 - Difference ≈ 0.0005 .
 - This is exactly half (0.5) of the nudge (0.001).

5. Summary & Takeaways

1. **Derivative = Slope:** It represents the rate of change of the output relative to a tiny change in the input.
 2. **Variable Slope:** For non-linear functions, the derivative value depends on the input variable (e.g., at $a = 2$ vs $a = 5$).
 3. **Lookup Tables:** You can find derivative formulas for standard functions in calculus textbooks or online. You don't need to derive them from scratch; you just need to know how to interpret them.
-

Computation Graph

1. Concept and Purpose

- **Definition:** The **Computation Graph** is a visual representation of the sequence of operations performed in a mathematical function or algorithm.
- **Role in Neural Networks:** It explains the organization of computations into two distinct phases:
 1. **Forward Propagation (Forward Pass):** Moving left-to-right to compute the final output (Cost Function J).
 2. **Backward Propagation (Backward Pass):** Moving right-to-left to compute derivatives (gradients) for optimization.

2. Illustrative Example

Consider a function J of three variables a, b, c :

$$J(a, b, c) = 3(a + bc)$$

To compute this via a graph, the function is broken down into distinct steps using intermediate variables (u, v):

1. **Step 1 (Compute u):** Calculate the product of b and c .
$$u = bc$$
2. **Step 2 (Compute v):** Add a to the intermediate result u .
$$v = a + u$$
3. **Step 3 (Compute J):** Multiply v by 3 to get the final output.
$$J = 3v$$

3. Numerical Verification (Forward Pass)

Using concrete values $a = 5, b = 3, c = 2$:

1. **Compute u :**

$$u = 3 \times 2 = 6$$

2. **Compute v :**

$$v = 5 + 6 = 11$$

3. **Compute J :**

$$J = 3 \times 11 = 33$$

$$(\text{Verification: } 3(5 + (3 \times 2)) = 3(5 + 6) = 3(11) = 33)$$

4. Summary of Flows

- **Left-to-Right (Blue Arrows):** Computes the value of the cost function J . This is **Forward Propagation**.
 - **Right-to-Left (Red Arrows):** Used to compute the derivatives of J with respect to inputs a, b, c . This is **Backward Propagation** (covered in the next section).
-

Derivatives with a Computation Graph

1. Concept: Backpropagation

- **Goal:** To calculate the derivative of the final output variable (usually the Cost Function J) with respect to various intermediate and input variables (v, a, u, b, c).
- **Direction:** Derivatives are computed **Right-to-Left** (opposite to the forward pass).
- **Terminology:** This process of going backward through the graph to compute derivatives is known as **Backpropagation**.

2. The Chain Rule

The core mathematical principle driving backpropagation is the **Chain Rule**.

- **Principle:** If variable a affects v , and v affects J , then the change in J given a change in a is the product of the individual changes along the path.
- **Formula:**
$$\frac{dJ}{da} = \frac{dJ}{dv} \cdot \frac{dv}{da}$$

3. Step-by-Step Derivative Calculation (Example)

Using the previous function $J = 3(a + bc)$ with inputs $a = 5, b = 3, c = 2$.

Step 1: Derivative w.r.t Final Intermediate (v)

- **Equation:** $J = 3v$
- **Logic:** If v increases by a tiny amount, J increases by 3 times that amount.
- **Result:** $\frac{dJ}{dv} = 3$

Step 2: Derivative w.r.t Input (a)

- **Path:** $a \rightarrow v \rightarrow J$
- **Equation:** $v = a + u$
- **Local Derivative:** $\frac{dv}{da} = 1$ (since slope of a in $a + u$ is 1).
- **Chain Rule:**
$$\frac{dJ}{da} = \frac{dJ}{dv} \cdot \frac{dv}{da} = 3 \cdot 1 = 3$$

Step 3: Derivative w.r.t Intermediate (u)

- **Path:** $u \rightarrow v \rightarrow J$
- **Equation:** $v = a + u$
- **Local Derivative:** $\frac{dv}{du} = 1$.
- **Chain Rule:**
$$\frac{dJ}{du} = \frac{dJ}{dv} \cdot \frac{dv}{du} = 3 \cdot 1 = 3$$

Step 4: Derivative w.r.t Input (b)

- **Path:** $b \rightarrow u \rightarrow v \rightarrow J$
- **Equation:** $u = bc$
- **Local Derivative:** $\frac{du}{db} = c = 2$ (since $u = 2b$ when $c = 2$).
- **Chain Rule:**
$$\frac{dJ}{db} = \frac{dJ}{du} \cdot \frac{du}{db} = 3 \cdot 2 = 6$$
- **Intuition:** If b bumps up by 0.001, u bumps up by 0.002 (twice as much). Since $\frac{dJ}{du} = 3$, J bumps up by $3 \times 0.002 = 0.006$. Total ratio is 6.

Step 5: Derivative w.r.t Input (c)

- **Path:** $c \rightarrow u \rightarrow v \rightarrow J$
- **Equation:** $u = bc$
- **Local Derivative:** $\frac{du}{dc} = b = 3$.
- **Chain Rule:**

$$\frac{dJ}{dc} = \frac{dJ}{du} \cdot \frac{du}{dc} = 3 \cdot 3 = 9$$

4. Implementation Notation (Code Convention)

When writing code (e.g., Python) for backpropagation, variable names are simplified to keep code clean.

- **Standard Notation:** $\frac{d\text{FinalOutputVar}}{d\text{var}}$
 - **Code Variable Name:** `dvar`
 - `dv` represents $\frac{dJ}{dv}$
 - `da` represents $\frac{dJ}{da}$
 - `db` represents $\frac{dJ}{db}$
 - **Usage:** In the code, you compute `dvar` values moving right-to-left, using previously computed gradients (e.g., using `du` to calculate `db`).
-

Logistic Regression Gradient Descent

1. Overview

- **Objective:** To compute the derivatives required to implement **Gradient Descent** for a Logistic Regression model.
- **Method:** Utilizing a **Computation Graph** to visualize the forward and backward propagation steps. While simple for logistic regression, this approach establishes the foundation for complex neural networks.

2. Computation Graph Representation

For a single training example with two features (x_1, x_2):

A. Forward Propagation (Computing Loss)

1. **Linear Combination (z):** Compute the weighted sum of inputs plus bias.

$$z = w_1x_1 + w_2x_2 + b$$
2. **Activation (a):** Apply the sigmoid function to get the prediction.

$$\hat{y} = a = \sigma(z)$$
3. **Loss Calculation (L):** Compute the loss between prediction a and true label y .

$$L(a, y) = -(y \log(a) + (1 - y) \log(1 - a))$$

B. Backward Propagation (Computing Derivatives)

To minimize the loss, we calculate derivatives moving from right to left (from Loss back to Parameters).

- **Derivative w.r.t Activation (da):**

- Represents $\frac{dL}{da}$.

- Formula:

$$da = -\frac{y}{a} + \frac{1-y}{1-a}$$

- **Derivative w.r.t Linear Output (dz):**

- Represents $\frac{dL}{dz}$.

- Derived via Chain Rule: $\frac{dL}{dz} = \frac{dL}{da} \cdot \frac{da}{dz}$.

- **Simplified Formula:**

$$dz = a - y$$

- *Note:* This simplification ($a - y$) is the standard implementation for logistic regression backpropagation.

- **Derivatives w.r.t Parameters (dw, db):**

- **Weights (dw_1, dw_2):**

$$dw_1 = x_1 \cdot dz$$

$$dw_2 = x_2 \cdot dz$$

- **Bias (db):**

$$db = dz$$

3. Parameter Updates (Single Example)

Once the gradients (dw_1, dw_2, db) are computed for a single example, the parameters are updated using the **Learning Rate** (α):

- $w_1 := w_1 - \alpha \cdot dw_1$
 - $w_2 := w_2 - \alpha \cdot dw_2$
 - $b := b - \alpha \cdot db$
-

Gradient Descent on m Examples

1. Cost Function & Derivatives Recap

To train the model on the entire training set, we define the **Cost Function** $J(w, b)$ as the average of the loss functions across all m training examples.

- **Cost Function:**

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(a^{(i)}, y^{(i)})$$

- **Derivative Rule:** The derivative of the overall cost function with respect to a parameter is simply the average of the derivatives of the individual losses.

$$\frac{\partial J(w, b)}{\partial w} = \frac{1}{m} \sum_{i=1}^m \frac{\partial L(a^{(i)}, y^{(i)})}{\partial w}$$

2. The Algorithm (Non-Vectorized Implementation)

To implement a **single step** of gradient descent across m examples using for loops:

Step A: Initialization

Initialize accumulators to zero.

- $J = 0$
- $dw_1 = 0, dw_2 = 0$ (assuming 2 features for this example)
- $db = 0$

Step B: Iteration over Training Set

For $i = 1$ to m :

1. Forward Pass:

- Compute linear output: $z^{(i)} = w^T x^{(i)} + b$
- Compute prediction: $a^{(i)} = \sigma(z^{(i)})$
- Accumulate Cost: $J+ = -[y^{(i)} \log(a^{(i)}) + (1 - y^{(i)}) \log(1 - a^{(i)})]$

2. Backward Pass (Compute Gradients):

- Compute error: $dz^{(i)} = a^{(i)} - y^{(i)}$
- Accumulate gradients for weights:
 - $dw_1+ = x_1^{(i)} dz^{(i)}$
 - $dw_2+ = x_2^{(i)} dz^{(i)}$
- Accumulate gradient for bias:
 - $db+ = dz^{(i)}$

Step C: Averaging

Divide accumulators by m to get the averages.

- $J/ = m$
- $dw_1/ = m$
- $dw_2/ = m$
- $db/ = m$

Step D: Parameter Update

Update parameters using the learning rate α .

- $w_1 := w_1 - \alpha \cdot dw_1$
- $w_2 := w_2 - \alpha \cdot dw_2$
- $b := b - \alpha \cdot db$

3. Implementation Challenges

- **Inefficiency:** The algorithm described above requires two distinct for loops:
 1. A loop over m **training examples**.
 2. A loop over n **features** (in the example above, $n = 2$, but in practice, n can be very large).
 - **The Problem with Loops:** In deep learning, where datasets are massive, explicit for loops are computationally slow and inefficient.
 - **Solution: Vectorization.** This set of techniques allows you to process entire matrices of data simultaneously, removing the need for explicit loops and significantly speeding up training.
-